RAM CHARAN VANGA
RA2511027010164
Sub ject : DSA

Unit 3, S5, SLO1 - Stack Applications - Infix to Postfix Conversion 2

Convert the following infix expression into postfix expression using the algorithm using stacks:
A – (B / C + (D % E * F) / G)* H

Answer: Conversion Trace:
Initial Stack: [( ], Append ')' to expression.
1. Scan 'A': Operand -> Postfix: A
2. Scan '-': Operator -> Stack: [(, -]
3. Scan '(': Left Paren -> Stack: [(, -, ( ]
4. Scan 'B': Operand -> Postfix: A B
5. Scan '/': Operator -> Stack: [(, -, (, /]
6. Scan 'C': Operand -> Postfix: A B C
7. Scan '+': Lower precedence than '/' -> Pop '/' -> Postfix: A B C / -> Push '+' -> Stack: [(, -, (, +]
8. Scan '(': Left Paren -> Stack: [(, -, (, +, ( ]
9. Scan 'D': Operand -> Postfix: A B C / D
10. Scan '%': Operator -> Stack: [(, -, (, +, (, %]
11. Scan 'E': Operand -> Postfix: A B C / D E
12. Scan '*': Same precedence as '%' (left-assoc) -> Pop '%' -> Postfix: A B C / D E % -> Push '*' -> Stack: [(, -, (, +, (, *]
13. Scan 'F': Operand -> Postfix: A B C / D E % F
14. Scan ')': Pop '*' -> Postfix: A B C / D E % F * -> Pop '(' -> Stack: [(, -, (, +]
15. Scan '/': Higher precedence than '+' -> Push '/' -> Stack: [(, -, (, +, /]
16. Scan 'G': Operand -> Postfix: A B C / D E % F * G
17. Scan ')': Pop '/' -> Postfix: ... G / -> Pop '+' -> Postfix: ... G / + -> Pop '(' -> Stack: [(, -]
18. Scan '*': Higher precedence than '-' -> Push '*' -> Stack: [(, -, *]
19. Scan 'H': Operand -> Postfix: A B C / D E % F * G / + H
20. Scan end ')': Pop '*' -> Postfix: ... H * -> Pop '-' -> Postfix: ... H * - -> Pop '(' -> Stack empty.

Final Postfix Expression: A B C / D E % F * G / + H * -